

Práctica 01 — Fundamentos de POO y Código Limpio (C++17)

1) Objetivos

- Aplicar **encapsulación, abstracción y polimorfismo** en C++.
- Escribir **código limpio**: nombres claros, funciones pequeñas, sin duplicación y con validación de invariantes.
- Emplear C++17 de forma responsable (excepciones, constexpr simples, = default).

2) Contenidos mínimos

- Pilares de POO (encapsulación, abstracción, polimorfismo).
- Buenas prácticas: *naming*, funciones cortas, evitar estados inválidos, manejo básico de errores con excepciones.
- Convenciones de estilo: llaves, sangría, límites de función (~20–30 líneas), comentarios solo cuando agreguen contexto.

3) Actividades (60–70 minutos)

1. (10') Configuración y repaso breve de POO.
2. (30') **Ejercicio A (obligatorio)**: clase Time segura con validación y formateo.
3. (20') **Ejercicio B (obligatorio)**: jerarquía polimórfica con Shape.

4) Ejercicio A — Clase Time segura y con formateo

Meta. Implementar una clase Time que:

- Garantice rangos válidos: $0 \leq \text{hour} \leq 23$, $0 \leq \text{minute}, \text{second} \leq 59$.
- Ofrezca métodos de lectura y modificación que **preserven invariantes**.
- Formatee en **militar** (HH:MM) y **estándar** (h:MM:SS AM/PM).

Requisitos.

- Atributos private.
- Constructores y *setters* que validen; lanzar `invalid_argument` ante datos inválidos.
- Métodos cortos, sin “números mágicos” (usa constantes internas).
- **Sin usar `std::`**: en el cliente: incluye `using namespace std`; en el archivo.

Plantilla base (C++17):

```
// practica01_time.cpp
#include <iostream>
#include <iomanip>
#include <sstream>
#include <stdexcept>
using namespace std;

class Time {
public:
    Time() = default;
    Time(int hour, int minute, int second) { set(hour, minute,
second); }

    // Observadores
    int hour() const noexcept { return hour_; }
    int minute() const noexcept { return minute_; }
    int second() const noexcept { return second_; }
```

Prof. Mg. Aldo Zanabria Gálvez

```
// Modificadores (validación estricta)
void hour(int h) { validate(h, minute_, second_); hour_ = h; }
void minute(int m) { validate(hour_, m, second_); minute_ = m; }
void second(int s) { validate(hour_, minute_, s); second_ = s; }

void set(int h, int m, int s) {
    validate(h, m, s);
    hour_ = h; minute_ = m; second_ = s;
}

string toMilitary() const {
    ostringstream os;
    os << setw(2) << setfill('0') << hour_ << ':'
        << setw(2) << setfill('0') << minute_;
    return os.str();
}

string toStandard() const {
    const int h12 = (hour_ == 0 || hour_ == 12) ? 12 : hour_ % 12;
    const char* ampm = (hour_ < 12) ? "AM" : "PM";
    ostringstream os;
    os << h12 << ':'
        << setw(2) << setfill('0') << minute_ << ':'
        << setw(2) << setfill('0') << second_ << ' ' << ampm;
    return os.str();
}

private:
    int hour_{0}, minute_{0}, second_{0};

    static void validate(int h, int m, int s) {
        if (h < 0 || h > 23) throw invalid_argument("hour out of range [0,23]");
        if (m < 0 || m > 59) throw invalid_argument("minute out of range [0,59]");
        if (s < 0 || s > 59) throw invalid_argument("second out of range [0,59]");
    }
};

int main() {
    try {
        Time t1(18, 30, 0);
        cout << t1.toMilitary() << " | " << t1.toStandard() << '\n';

        Time t2; // 00:00:00
        t2.minute(5);
        t2.second(7);
        cout << t2.toMilitary() << " | " << t2.toStandard() << '\n';

        // Prueba inválida (descomentar para verificar excepción):
        // Time bad(29, 73, 0);
    }
}
```

Prof. Mg. Aldo Zanabria Gálvez

```
    } catch (const exception& ex) {  
        cerr << "Error: " << ex.what() << '\n';  
        return 1;  
    }  
    return 0;  
}
```

Criterios de evaluación (Ejercicio A).

- Encapsulación y validación consistente.
- Métodos breves y nombres que revelen intención.
- Formateo correcto en ambos estilos.

5) Ejercicio B — Polimorfismo con Shape

Meta. Definir una interfaz Shape con area() y perimeter(). Implementar Rectangle y Circle, y procesar objetos vía punteros polimórficos.

Requisitos.

- Shape con **métodos virtuales puros**.
- Uso de **unique_ptr** y un contenedor de **polimorfismo** (vector).
- Evitar *números mágicos* en producción (en pruebas, documentarlos).

Plantilla base (C++17):

```
// practica01_shape.cpp  
#include <iostream>  
#include <vector>  
#include <memory>  
#include <cmath>  
using namespace std;  
  
struct Shape {  
    virtual ~Shape() = default;  
    virtual double area() const = 0;  
    virtual double perimeter() const = 0;  
};  
  
class Rectangle : public Shape {  
    double w_, h_;  
public:  
    Rectangle(double w, double h) : w_(w), h_(h) {}  
    double area() const override { return w_ * h_; }  
    double perimeter() const override { return 2 * (w_ + h_); }  
};  
  
class Circle : public Shape {  
    double r_;  
public:  
    explicit Circle(double r) : r_(r) {}  
    double area() const override { return M_PI * r_ * r_; }  
    double perimeter() const override { return 2 * M_PI * r_; }  
};
```

Prof. Mg. Aldo Zanabria Gálvez

```
int main() {  
    vector<unique_ptr<Shape>> shapes;  
    shapes.emplace_back(make_unique<Rectangle>(4.0, 3.0));  
    shapes.emplace_back(make_unique<Circle>(2.5));  
  
    for (const auto& s : shapes) {  
        cout << "A=" << s->area() << " | P=" << s->perimeter() <<  
'\n';  
    }  
    return 0;  
}
```

Criterios de evaluación (Ejercicio B).

- Polimorfismo correcto y seguro (destructores virtuales, override).
- Diseño simple y coherente (SRP).
- Claridad del *main* y salida verificable.

6) Entregables

- Repositorio practica01-poo2-<apellido> con:
 - practica01_time.cpp y practica01_shape.cpp (compilables con C++17).
 - README.md (cómo compilar: g++ -std=gnu++17 -O2 -Wall archivo.cpp -o app).
 - **Informe breve (1–2 páginas):**
 - Explicación línea a línea de **un método clave** por ejercicio.
 - Decisiones de diseño (encapsulación, validación, nombres).
 - Corrección ortográfica y coherencia semántica.

7) Rúbrica (100%)

- **Time seguro y formateo** (30%).
- **Jerarquía polimórfica funcional** (30%).
- **Calidad de código** (nombres, funciones cortas, sin duplicación, comentarios mínimos) (20%).
- **Informe y README** (claridad, ortografía, semántica) (20%).

8) Trabajo de investigación (para casa, 20–30 min)

- **“Composición sobre herencia”**: redacta 1 página con un ejemplo breve en C++ (clases que delegan responsabilidades a componentes). Indica **ventajas de mantenibilidad** y un caso en el que **sí** usar herencia.